

Lab0 实验报告：Git 版本控制基础

2026 年 9 月

Lab0 实验报告：Git 版本控制基础

一、问题回答

1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

没有过多人的协同开发的经历。之前的编程作业都是独立完成，代码管理也比较简单，基本就是在本地直接修改文件，偶尔通过 U 盘或聊天工具跟同学交换代码。这种方式一个人用还行，但多人协作的话很容易出现版本混乱、互相覆盖的问题。这次实验算是第一次接触版本控制，对分支和合并有了初步的了解，以后如果要参与团队项目，应该会用到这些知识。

2. 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

刚开始觉得 `git add` 和 `git commit` 分成两步有点麻烦，后来理解了一下，主要有这几个原因：

一是选择性提交。实际开发中经常同时改了好几个文件，但这些改动可能属于不同的功能。比如修了一个 bug，同时又写了新功能，这两件事逻辑上没关系，应该分开提交。暂存区就是让你选择哪些改动放进这次提交，不用把所有修改一股脑提交上去。

二是提交前可以检查。用 `git diff --cached` 能看到这次提交具体包含哪些改动，确认没问题再 `commit`，避免提交了一些不该提交的内容。

三是让提交历史更清晰。每次提交最好只包含一个有意义的改动，这样以后查历史、回滚都方便。暂存区帮助开发者把零散的修改整理成有逻辑的提交。

总的来说，暂存区相当于一个“待提交清单”，让开发者对每次提交的内容有控制权。

3. `git branch` 和 `git branch -a` 的区别是什么？

`git branch` 只列出本地仓库的分支。比如本地有 `main` 和 `feature` 两个分支，它就显示这两个，当前分支前面会标 `*`。

`git branch -a` 会同时列出本地分支和远程分支。远程分支以 `remotes/origin/` 为前缀，比如 `remotes/origin/main`、`remotes/origin/HEAD`。

区别就是 `-a` (all) 参数会把远程跟踪分支也显示出来。团队协作时，别人推送了新分支到远程，用 `git branch` 看不到，需要加 `-a` 才能看到。

二、实验步骤

2.1 环境准备

安装 Git 后配置用户名和邮箱：

```
git config --global user.name " 你的名字"
git config --global user.email " 你的邮箱"
```

Fork 模板仓库并 Clone 到本地：

```
git clone < 你的仓库地址 >
cd lab0
```

2.2 初始提交

模板仓库中的 `main.c` 内容如下：

```
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("Hello, world!\n");
}
```

修改输出内容后提交推送：

```
git add main.c
git commit -m "Change printed message from 'Hello, world!' to 'I see you!'"
git push origin main
```

2.3 创建 feature 分支并修改

创建 **feature** 分支：

```
git checkout -b feature
```

将 printf 内容改为 "from feature\n":

```
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("from feature\n");
}
```

提交推送:

```
git add main.c
git commit -m "feat: change message on feature branch"
git push origin feature
```

2.4 在 main 分支上制造冲突

切回 main 分支, 对同一行做不同修改:

```
git checkout main
```

将 printf 改为 "from main\n":

```
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("from main\n");
}
```

提交推送:

```
git add main.c
git commit -m "feat: change message on main branch"
git push origin main
```

2.5 合并分支并解决冲突

将 `feature` 分支合并到 `main`:

```
git merge feature
```

出现合并冲突:

Auto-merging `main.c`

CONFLICT (content): Merge conflict in `main.c`

Automatic merge failed; fix conflicts and then commit the result.

打开 `main.c`, 内容变为:

```
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
<<<<<<< HEAD (Current Change)
    printf("from main\n");
=====
    printf("from feature\n");
>>>>>>> feature (Incoming Change)
}
```

<<<<<<< HEAD 和 ===== 之间是 `main` 分支的改动, ===== 和 >>>>>>> feature 之间是 `feature` 分支的改动。我保留了 `main` 分支的内容, 删除冲突标记和 `feature` 分支的内容, 修改后:

```
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    printf("from main\n");
}
```

提交合并结果:

```
git add main.c
```

```
git commit -m "Merge branch 'feature'"
git push origin main
```

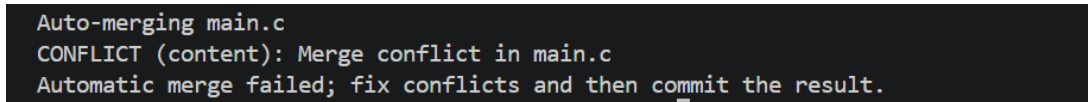
2.6 查看提交历史

```
*   dadaed7 (HEAD -> main) Merge branch 'feature'
|\
| * 572362e (feature) feat: change message on feature branch
* | f224e99 feat: change message on main branch
|/
*   f2b02cc (origin/main, origin/HEAD) Change printed message from 'Hello, world!' to 'I see
*   49657eb Initial commit
```

从提交历史可以看出，`main` 和 `feature` 从同一个节点分叉，各自提交后在 `main` 上合并，形成了一个 merge commit。

三、冲突解决截图

截图 1: `feature` 分支上的修改



```
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

Figure 1: `feature` 分支修改

在 `feature` 分支上将输出改为 `"from feature\n"`。

截图 2: 合并冲突提示

执行 `git merge feature` 后出现冲突，自动合并失败。

截图 3: 冲突标记

编辑器中显示冲突标记，HEAD 侧为 `main` 分支的改动，`feature` 侧为 `feature` 分支的改动。

截图 4: 解决冲突后的文件

删除冲突标记，保留 `main` 分支的内容。

截图 5: 合并后的 Git 提交历史

`git log --oneline --all --graph` 的结果，分支分叉与合并过程清晰可见。

```

1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      <<<<<<< HEAD (Current Change)
7      printf("from main\n");
8      =====
9      printf("from feature\n");
10     >>>>>> feature (Incoming Change)
11 }
12

```

Figure 2: 合并冲突

```

1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6
7      printf("from main\n");
8
9
10 }
11

```

Figure 3: 冲突标记

```

*   dadaed75bc9e43d7f7072e43a17497ced2da744b (HEAD -> main) Merge branch 'feature'
|
| *   572362e9a563a05590bab9c98d1f776cf07b0c9b (feature) feat: change message on feature branch
| *   f224e990c7c836265f93da987d58d19546c75fe2 feat: change message on main branch
|/
*   f2b02cc70fb516fc98a338d8d0022e209a3a2ec7 (origin/main, origin/HEAD) Change printed message from 'Hello, world!' to 'I see you!'
*   49657ebc140e02af60c041c8807f4b0519da71e2 Initial commit

```

Figure 4: 解决冲突



```
C main.c
lab0 > C main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("from feature\n");
7  }
8
```

Figure 5: git log

四、学习资源总结（选做）

资源一：Conventional Commits 规范

阅读了 Conventional Commits 规范。

该规范为 commit message 定义了一种格式：类型（作用域）：描述。类型包括 **feat**（新功能）、**fix**（修复）、**docs**（文档）等。例如 **feat(auth): add login validation** 从提交信息就能看出这次改动的内容和范围。

之前写 commit message 比较随意，经常写“update”或“fix bug”，过一段时间就忘了具体改了什么。使用这个规范后，提交历史会更加清晰，也方便自动生成 CHANGELOG。

资源二：Git Flow 工作流

阅读了 Vincent Driessen 的 A successful Git branching model。

Git Flow 将分支分为几种类型：**main** 存放正式代码，**develop** 作为开发主线，新功能在 **feature/*** 分支上开发，发布前使用 **release/*** 分支，线上问题通过 **hotfix/*** 分支修复。

这种分支模型适合有固定发布周期的项目。小项目可能不需要这么复杂的流程，但对于团队协作来说，明确的分支策略确实能减少很多混乱。

对“为什么要学习 Git”的理解

完成这次实验后，对 Git 的作用有了更直观的认识。

首先是版本管理。以前不用 Git 的时候，代码改坏了就很难恢复。Git 提供了完整的历史记录，可以随时回退到任意版本，相当于给代码加了一份保险。

其次是协作。多人同时修改同一个文件时，如果没有版本控制，很容易互相覆盖。Git 的分支机制让每个人可以独立开发，最后通过合并整合代码，即使出现冲突也有明确的解决方式。

最后是行业需求。无论是开源项目还是企业开发，Git 都是主流的版本控制工具。掌握 Git 基本操作是参与软件开发的基础。

五、实验总结

这次实验完成了一个完整的 Git 工作流程：Fork 仓库、修改代码、创建分支、制造并解决冲突。之前对 Git 的了解主要停留在 `git add`、`git commit`、`git push` 这几个基本命令，这次实际遇到了合并冲突并手动解决，对分支管理和合并机制有了更具体的理解。

冲突产生的原因是两个分支修改了同一行代码，Git 无法自动判断应该保留哪个版本，需要开发者手动选择。解决过程并不复杂，关键是理解了冲突标记的含义和分支合并的基本原理。